Q3 - Engineer a Device-Status Register with Bitwise Operators (6 marks)

3.1 Bit operations on one unsigned char (bit i = device i; mask = 1u << dev)

Task	Expression	Idea
Switch ON	reg | (1u << dev)	OR with the mask forces that bit to 1; other bits unchanged
Switch OFF	reg & ~(1u << dev)	~mask has 0 only at the device bit; AND clears it, keeps the rest
Toggle	reg ^ (1u << dev)	XOR flips the bit (0 to 1, 1 to 0)
Test	(reg >> dev) & 1u	Brings the bit to position 0 and isolates it, giving exactly 0 or 1
Print in binary	((reg >> bit) & 1u) for bit = 7 down to 0	Test each bit from the most significant
Count devices ON	sum of (reg >> bit) & 1u over 8 bits	Or Kernighan's method: reg &= reg - 1 clears the lowest set bit each loop; loop count = number of ones

3.2 Completed code (TODO 1 to TODO 4) and execution

q3.c

```
#include <stdio.h>

/* Each bit of one byte stores the ON/OFF state of one device. */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)
{
    return (unsigned char)(reg | (1u << dev));      /* TODO 1: set bit   */
}

unsigned char switch_off(unsigned char reg, int dev)
{
    return (unsigned char)(reg & ~(1u << dev));     /* TODO 2: clear bit */
}

unsigned char toggle(unsigned char reg, int dev)
{
    return (unsigned char)(reg ^ (1u << dev));      /* TODO 3: flip bit  */
}

int is_on(unsigned char reg, int dev)
{
    return (reg >> dev) & 1u;                       /* TODO 4: test bit  */
}

int count_on(unsigned char reg)                     /* devices that are ON */
{
    int n = 0;
    for (int bit = 0; bit < 8; bit++) n += (reg >> bit) & 1u;
    return n;
}

void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--) putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}

int main(void)
{
```

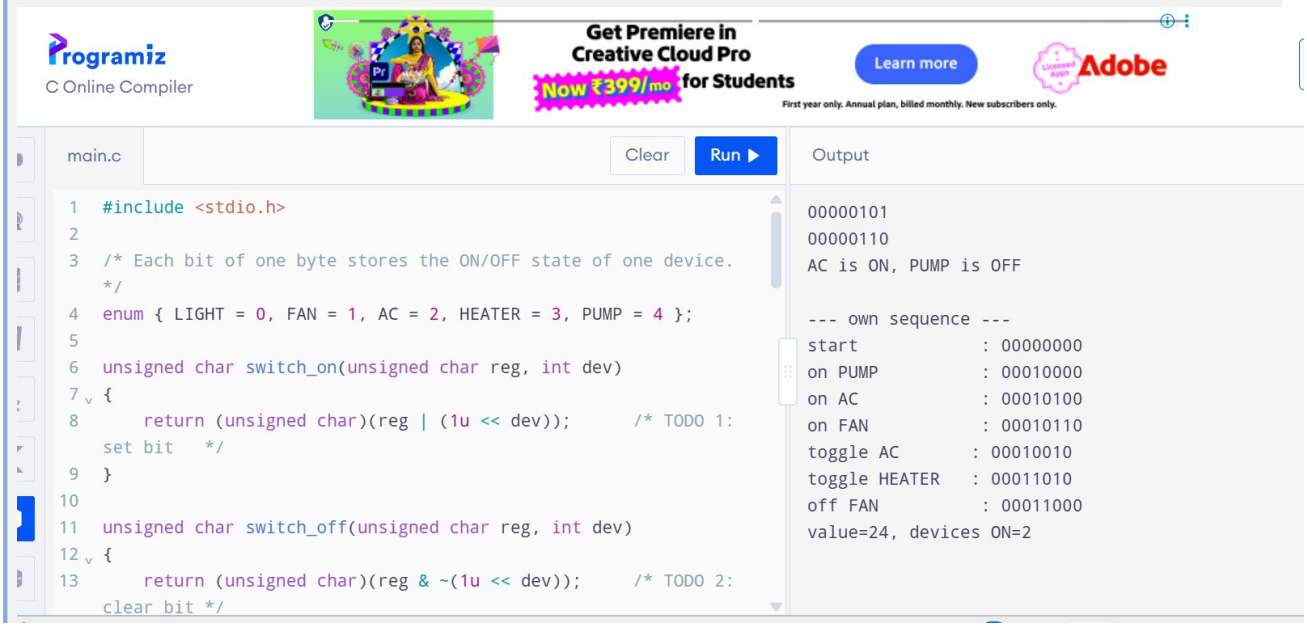
```
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s\n", is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");

    /* My own sequence */
    printf("\n--- own sequence ---\n");
    reg = 0;                    printf("start          : "); show(reg);
    reg = switch_on(reg, PUMP);   printf("on PUMP        : "); show(reg);
    reg = switch_on(reg, AC);     printf("on AC          : "); show(reg);
    reg = switch_on(reg, FAN);    printf("on FAN         : "); show(reg);
    reg = toggle(reg, AC);        printf("toggle AC      : "); show(reg);
    reg = toggle(reg, HEATER);    printf("toggle HEATER  : "); show(reg);
    reg = switch_off(reg, FAN);   printf("off FAN        : "); show(reg);
    printf("value=%u, devices ON=%d\n", reg, count_on(reg));
    return 0;
}
```

Observed output

```
00000101
00000110
AC is ON, PUMP is OFF

--- own sequence ---
start          : 00000000
on PUMP        : 00010000
on AC          : 00010100
on FAN         : 00010110
toggle AC      : 00010010
toggle HEATER  : 00011010
off FAN        : 00011000
value=24, devices ON=2
```



The first three lines match the expected output given in the question exactly.

3.3 Register value after each statement in main() (predicted before running, then verified)

Statement	Decimal	Binary (predicted)	Printed?	Observed
unsigned char reg = 0;	0	00000000	-	matches

Statement	Decimal	Binary (predicted)	Printed?	Observed
reg = switch_on(reg, LIGHT);	1	00000001	-	matches
reg = switch_on(reg, AC);	5	00000101	show()	00000101
reg = toggle(reg, FAN);	7	00000111	-	matches
reg = switch_off(reg, LIGHT);	6	00000110	show()	00000110
is_on(reg, AC) / is_on(reg, PUMP)	6	bit 2 = 1, bit 4 = 0	printf	AC ON, PUMP OFF

My own sequence (predicted first, then verified by the program):

Operation	Decimal	Binary	Devices ON
start	0	00000000	none
switch_on PUMP (bit 4)	16	00010000	PUMP
switch_on AC (bit 2)	20	00010100	AC, PUMP
switch_on FAN (bit 1)	22	00010110	FAN, AC, PUMP
toggle AC	18	00010010	FAN, PUMP
toggle HEATER (bit 3)	26	00011010	FAN, HEATER, PUMP
switch_off FAN	24	00011000	HEATER, PUMP (count = 2)

Observed output for this sequence (see above) matched every prediction, ending with value = 24 and count_on = 2.

3.4 Inquiry

Why `reg & 1 << dev` and `(reg & 1) << dev` differ. The shift operators have higher precedence than &, so reg & 1 << dev means reg & (1 << dev): it masks the bit of device dev - a correct test, but the result is the mask value (1 << dev, e.g. 4 for AC), not 1, so a comparison such as == 1 fails for every device except dev 0. In (reg & 1) << dev the brackets force the AND first: it examines only bit 0 (the LIGHT), and then shifts that single bit to position dev. It tells you the state of device 0, whichever device you asked about. Observed with dev = 2:

```
reg=0x04 dev=2:  reg & 1 << dev = 4   (reg & 1) << dev = 0
reg=0x05 dev=2:  reg & 1 << dev = 4   (reg & 1) << dev = 4
reg=0x01 dev=2:  reg & 1 << dev = 0   (reg & 1) << dev = 4
```

- reg = 0x04 (AC on, LIGHT off): reg & 1 << dev = 4 (correct, AC is on) but (reg & 1) << dev = 0 (wrongly says AC is off).
- reg = 0x01 (only LIGHT on): reg & 1 << dev = 0 (correct) but (reg & 1) << dev = 4 (wrongly says AC is on).
- reg = 0x05: both give 4 only by coincidence.

What goes wrong with a signed char when bit 7 is set. Bit 7 becomes the sign bit. 0x80 does not fit in a signed char; the conversion is implementation-defined and normally gives -128. In expressions the value is promoted to int with sign extension, so it becomes 0xFFFFFF80. Observed:

```
unsigned: value=128  hex=80  >>1 = 40  ==0x80? 1
signed  : value=-128 hex=ffffff80  >>1 = fffffc0  ==0x80? 0
signed 0x80, while(t) t>>=1 : still non-zero after 20 shifts (t=-1)
unsigned 0x80 same loop      : ends after 8 shifts
```

- Printing shows -128 (and ffffff80 in hex) instead of 128 (80); reg == 0x80 is false.
- Right shift of a negative value is implementation-defined and usually arithmetic (sign-filling): reg >> 1 gives 0xFFFFFFC0 instead of 0x40, which corrupts masks and tests.
- A loop like while (reg) reg >>= 1; (e.g. for counting bits) never ends for a negative value, because it stays -1; the unsigned version ends after 8 shifts.
- Left shifting a negative value (reg << 1) is undefined behaviour in C.

The show() function appears to work with a signed char only because of the & 1u mask. The register should always be unsigned char (or uint8_t) so that all 8 bits are plain data.

3.5 Alternative concept and complexity analysis - `char state[8]`

With one array element per device the operations become state[dev] = 1; (on), state[dev] = 0; (off), state[dev] = ! state[dev]; or state[dev] ^= 1; (toggle), and state[dev] (test). No masks or shifts are needed.

Aspect	Bit-packed unsigned char	Array char state[8]
Set / clear / toggle / test one device	O(1): a shift plus one bitwise operation	O(1): one indexed memory access
Count devices ON	Loop over 8 bits, O(n) with n = 8 devices (a constant); Kernighan's method needs one pass per ON device; a 256-entry lookup table or hardware popcount makes it O(1)	Loop over 8 elements adding them, O(n) with n = 8 devices (a constant); no shortcut
Space per home	1 byte	8 bytes
Space for 1,000,000 homes	1,000,000 bytes (about 0.95 MiB)	8,000,000 bytes (about 7.63 MiB) - 8 times more
Other points	Whole state can be copied, compared (reg1 == reg2), stored or sent as one byte; many devices can be changed at once with a mask; code is harder to read and bugs like the precedence/signedness errors above are possible	Very readable and simple to debug; elements can store more than ON/OFF (e.g. fan speed); wastes memory, which matters on a low-cost controller

Conclusion: both designs take O(1) time for single-device operations and O(number of devices) for counting, so time does not decide the choice. Space does: for a million homes the bit-packed register saves about 7 MB and is the right design for a memory-limited controller, provided an unsigned type and careful bracketing of masks are used.